Training Runbook -- RTX 5090

Operational guide for executing the revision training run on a single RTX 5090.

Prepared: 2026-06-05

Scope. This runbook covers the software stack, vectorisation strategy, training matrix, reproducibility, monitoring, evaluation, and troubleshooting needed to execute the 40-run revision training matrix on a single RTX 5090 workstation.

Locked choices (from prior planning): paper-stated hyperparameters (lr = 9×10^{-5}, ent_coef = 0.05, goal = +100); 15 primary runs ($N\in\{2,3,4,5\}$, 3 seeds, 100k steps + N=5 at 25×25 with 150k steps) and 25 reward-sensitivity runs (5 params × 5 levels, 50k steps).

Hardware target. NVIDIA GeForce RTX 5090 (Blackwell, sm_120, 32 GB VRAM, ~1700 FP16 TFLOPS). The 5090 is severely underutilised by single-environment PPO on a 20×20 grid -- the rollout is bottlenecked on Python env stepping, not GPU compute. Section 4 explains how to exploit this.

1. RTX 5090 software stack

The RTX 5090 uses Blackwell architecture with compute capability sm_120. Older PyTorch wheels (cu118, cu121, cu124) do **not** include sm_120 kernels, so training will either fail with "no kernel image is available" or silently fall back to CPU. You must use the cu128 (CUDA 12.8) wheel of PyTorch 2.7 or newer.

1.1 Required components

Component	Required version	Notes
NVIDIA driver	>= 565.x	Driver must support CUDA 12.8 runtime
CUDA Toolkit	12.8 (recommended) or 12.4	12.4 will work but is slower; 12.8 is native
Python	3.10 - 3.12	3.13 not yet supported by SB3 stable
PyTorch	2.7.0 or newer, cu128 wheel	Older wheels lack sm_120 kernels
torchvision / torchaudio	matched to torch 2.7+	Only needed if you import them
Stable-Baselines3	>= 2.4.0	Required for gymnasium API and current PPO impl.
Gymnasium	>= 0.29	Replaces deprecated openai gym
NumPy	1.26.x or 2.x	SB3 2.4+ supports both
TensorBoard	>= 2.15	For training-curve logging

1.2 One-command install (Windows PowerShell)

From inside your project's Python environment:

```
pip install --upgrade pip
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu128
pip install stable-baselines3 gymnasium tensorboard pandas matplotlib scipy
```

1.3 Stack validation script

Save the following as `scripts/check_stack.py` and run it *before* launching any training. If any check fails, do not proceed -- you will burn hours on a stack that silently runs on CPU.

```
import sys, torch, numpy as np
import stable_baselines3 as sb3, gymnasium as gym
print(f"python : {sys.version.split()[0]}")
print(f"numpy : {np.__version__}")
print(f"torch : {torch.__version__}")
print(f"cuda : {torch.version.cuda}")
print(f"sb3 : {sb3.__version__}")
print(f"gymnasium: {gym.__version__}")
assert torch.cuda.is_available(), "CUDA not available"
assert torch.cuda.device_count() >= 1, "No CUDA device"
name = torch.cuda.get_device_name(0)
cap = torch.cuda.get_device_capability(0)
print(f"gpu : {name} (sm_{cap[0]}{cap[1]})")
assert cap == (12, 0), f"Expected sm_120 (Blackwell), got sm_{cap[0]}{cap[1]}"
x = torch.randn(2048, 2048, device="cuda")
y = (x @ x).sum().item()
print(f"matmul OK on cuda: y={y:.2f}")
print("STACK OK")
```

Expected output: torch 2.7+, cuda 12.8, sb3 2.4+, gymnasium 0.29+, gpu "NVIDIA GeForce RTX 5090 (sm_120)", and a clean STACK OK line.

2. Project layout and new files

All new code lives in new files; legacy paper-cited scripts (`v*.py`, `rl_*.py`, `map_gen_*.py`) are not modified.

```
d:/RL/Teal_rl/RL/RL_V2.0/
  env_revision.py # Clean RectangleReductionEnv with paper Table III
  map_revision.py # Seedable wrapper around map_gen_v5
  train_revision.py # PPO training driver (single run)
  astar_baseline.py # A* + meeting-point heuristic (baseline)
  eval_revision.py # PPO vs A* rollouts -> results.csv
  plots_revision.py # CSV -> paper figures
  run_all_revision.ps1 # End-to-end PowerShell driver
  scripts/
    check_stack.py # Pre-flight validation
    monitor_runs.ps1 # nvidia-smi + log tail
  logs_revision/ # Per-run TensorBoard + config.json + model
  results_revision/ # Aggregated CSV + figures
  reward_sensitivity.py # (already exists -- minor CLI touch-up only)
```

3. Hyperparameters and reward table (frozen)

3.1 PPO hyperparameters

Parameter	Value	Notes
algorithm	PPO (Stable-Baselines3)	
policy	MultiInputPolicy	Dict observation: {known_map, robot_positions}
net_arch	[64, 64]	Two hidden FC layers, tanh
learning_rate	9e-5	Paper III-C.2 (locked)
ent_coef	0.05	Paper III-C.1 (locked)
n_steps	2048	Rollout buffer per env
batch_size	64	Minibatch for PPO update
n_epochs	10	Update epochs per rollout
gamma	0.99	Discount factor
gae_lambda	0.95	GAE smoothing
clip_range	0.2	PPO clip ratio epsilon
vf_coef	0.5	Value-function loss weight
max_grad_norm	0.5	Gradient clipping
seed	0, 1, 2	Three seeds per primary config
total_timesteps	100k or 150k or 50k	See Section 5 matrix
device	cuda	Sub-ms inference on 5090

3.2 Reward table (locked, paper Table III)

Event	Reward
Bounding area decreases, still above threshold	+20
Bounding area decreases below threshold (goal)	+100
Bounding area increases	-0.5
Collision with wall or obstacle	-5
Collision with another robot	-5

4. Vectorisation strategy on the 5090

This is the most important RTX-5090-specific section. Naive single-environment PPO on a tiny grid uses about 5-10% of the 5090. Two layers of parallelism are available and you should use both.

4.1 Layer 1: vectorised environments inside a single run

Use `SubprocVecEnv` to step many copies of the environment in parallel within one training process. PPO already supports this natively: each call to `env.step` aggregates rollouts across all worker processes before passing them to the GPU.

- For a 20×20 grid with $N\in\{2,3,4,5\}$, the right number of parallel envs is **16**. This saturates the 16 P-cores of a typical 5090 host CPU; beyond 16 you start spawning extra cores that contend with each other.
- Memory footprint per env is < 10 MB, so 16 envs cost < 200 MB host RAM total.
- Wall-clock speed-up: about 8× on average vs. a single env, because Python env stepping is the bottleneck, not GPU compute.

Code skeleton:

```
from stable_baselines3 import PPO
from stable_baselines3.common.vec_env import SubprocVecEnv
from env_revision import make_env

env = SubprocVecEnv([make_env(n_robots=4, map_size=20, seed=i) for i in range(16)])
model = PPO("MultiInputPolicy", env, n_steps=2048, batch_size=64,
    learning_rate=9e-5, ent_coef=0.05, device="cuda")
model.learn(total_timesteps=100_000)
```

4.2 Layer 2: multiple training runs in parallel processes

After Layer 1, a single config still uses only a small fraction of the 32 GB VRAM. You can run multiple training configurations on the same GPU as separate Python processes.

- Each PPO process on this network architecture uses about **1-1.5 GB of VRAM**; the 5090's 32 GB therefore comfortably holds 8-16 concurrent runs.
- **Recommended:** run 4 concurrent processes, each with 16 vectorised envs. This uses about 6 GB VRAM, 64 worker processes, and keeps the GPU at 60-80% utilisation. Going to 8 concurrent processes maxes the host CPU before it maxes the GPU.
- Always pass a distinct `--seed` and `--tag` to each process so logs and checkpoints do not collide.

Run 4 in parallel via PowerShell (each --& backgrounds the job):

```
Start-Process python -ArgumentList "train_revision.py --n-robots 4 --map-size 20 --seed 0
--steps 100000 --tag primary"
Start-Process python -ArgumentList "train_revision.py --n-robots 4 --map-size 20 --seed 1
--steps 100000 --tag primary"
Start-Process python -ArgumentList "train_revision.py --n-robots 4 --map-size 20 --seed 2
--steps 100000 --tag primary"
Start-Process python -ArgumentList "train_revision.py --n-robots 5 --map-size 20 --seed 0
--steps 100000 --tag primary"
Wait-Process -Name python
```

4.3 What *not* to do

- Do **not** use DataParallel / DistributedDataParallel -- the policy network is tiny (under 100 kB of parameters); multi-GPU sharding wastes more on synchronisation than it saves.

- Do **not** set `device="cpu"` "because the env is small". The PPO gradient step is still substantial, and CPU PPO is roughly 3× slower on this workload than cuda PPO with 16 envs.
- Do **not** increase `n_steps` beyond 4096 -- the rollout buffer scales linearly in memory, and the gradient variance reduction is marginal past 2048.

5. Training matrix with RTX 5090 timing

5.1 Primary runs (15 total)

Config	N	Map	Seeds	Steps	Wall (1 env)	Wall (16 env, 5090)
C1	2	20x20	0,1,2	100k	~45 min	~6 min
C2	3	20x20	0,1,2	100k	~50 min	~7 min
C3	4	20x20	0,1,2	100k	~55 min	~8 min
C4	5	20x20	0,1,2	100k	~60 min	~9 min
C5	5	25x25	0,1,2	150k	~110 min	~16 min

Total primary wall-clock with 16-env vectorisation, sequential runs: (6+7+8+9)×3 + 16×3 = 90 + 48 = **138 min** ≈ **2.3 h**.
With 4 concurrent processes: ≈ **35-40 min**.

5.2 Reward-sensitivity sweep (25 runs)

Each run: N=4, 20×20, single seed, 50k steps. Same 16-env vectorisation. Per-run wall-clock: ~4 min. Sequential total: ~100 min. With 4 parallel processes: ~25 min. Use the existing `reward_sensitivity.py` with a small CLI touch-up to accept --n-envs.

5.3 Total time budget

Phase	Sequential (16-env)	4 parallel (16-env each)
Primary (15 runs)	~2.3 h	~40 min
Sensitivity (25 runs)	~1.7 h	~25 min
Evaluation rollouts (PPO + A*)	~1 h	~1 h (CPU-bound)
Plot generation	~5 min	~5 min
Total	~5 h	~1.2 h

Either column fits in a single working session. The 4-parallel column requires a 16-core CPU for full speed-up. If your host has 8 cores, run 2 concurrent processes and budget accordingly.

6. Reproducibility checklist

Every training run writes a `config.json` next to its checkpoint so the exact reproduction conditions are preserved.

6.1 Seed every randomness source

```
import random, numpy as np, torch
def set_seed(seed: int):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    # env seeding via env.reset(seed=seed) inside make_env
    # stable-baselines3 also exposes set_random_seed
```

6.2 config.json schema (written by train_revision.py)

```
{
  "git_commit": "<sha>",
  "torch": "2.7.0+cu128",
  "sb3": "2.4.0",
  "gpu": "NVIDIA GeForce RTX 5090",
  "n_robots": 4, "map_size": 20, "seed": 0,
  "total_timesteps": 100000, "n_envs": 16,
  "learning_rate": 9e-5, "ent_coef": 0.05,
  "reward": {"area_dec":20,"area_inc":-0.5,"collide":-5,"goal":100},
  "wall_clock_sec": 482,
  "final_eval_success_rate": 0.94
}
```

6.3 What to commit to git

- All new `*_revision.py` files.
- `logs_revision/<run>/config.json` for every run (small text files; commit them).
- `results_revision/results.csv` -- the aggregated evaluation table.
- Generated figures in `results_revision/figures/`.
- **Do not** commit the SB3 model zip files -- they are large; keep them under .gitignore.

7. Monitoring during training

7.1 TensorBoard

Each run writes to `logs_revision/<tag>/<config>/<seed>/tb/`. Launch TensorBoard once at the parent directory:

```
tensorboard --logdir logs_revision --port 6006
```

Curves to watch: `rollout/ep_rew_mean` (should rise then plateau), `rollout/ep_len_mean` (should fall), `train/value_loss` (should fall), and `train/entropy_loss` (should stabilise; if it collapses to near-zero too early, policy is over-exploiting).

7.2 GPU utilisation

In a separate PowerShell window:

```
while ($true) { Clear-Host; nvidia-smi; Start-Sleep -Seconds 2 }
```

Healthy signs: GPU-Util > 40% (single run) or > 70% (4 parallel runs); GPU memory occupied; one Python process per concurrent run. If GPU-Util < 5%, training is probably stuck on CPU -- re-run the stack check (Section 1.3).

7.3 Per-run progress.json

`train_revision.py` writes a tiny `progress.json` every 5k steps so you can sanity-check long runs without opening TensorBoard.

7.4 Crash recovery

Save a checkpoint every 25k steps via SB3 `CheckpointCallback`. If a process dies, resume from the latest checkpoint rather than restarting -- this preserves the seed reproducibility of the final model.

8. Evaluation pipeline

8.1 Inputs

- Trained policies in `logs_revision/<tag>/.../model.zip`.
- Test-case maps from `Maps/` (the 5 maps used in Sections IV-A through IV-E of the paper) + the same maps regenerated with seeds 0-19 for statistical aggregation.
- A* baseline implementation in `astar_baseline.py`.

8.2 What eval_revision.py does

- Loads each trained policy and rolls it out for 20 episodes per map.
- For the same map seeds, runs the A* rendezvous baseline.
- Logs per-rollout: total_distance, max_distance, success, steps_to_converge, inference_time_ms.
- Emits a single `results.csv` with one row per (method, config, map, seed).

8.3 results.csv schema

```
method,config,n_robots,map_size,test_case,map_idx,seed,
  success,steps,total_distance,max_distance,inference_ms
```

8.4 Statistical reporting

- Mean ± std per (method, config) over all seeds.
- Paired Wilcoxon signed-rank test for PPO vs A* on each metric per map.
- Bootstrap (1000 samples) confidence intervals on the scaling curve (Fig. 6 in the paper).
- All statistical tests use `scipy.stats`.

9. Figures and tables to produce

Output	Source	Paper section it replaces
Fig. 4 (training curves with std bands)	TensorBoard logs from primary runs	Section III-C, Fig. 4 (existing)
Fig. 6 (scaling bar chart with error bars)	results.csv aggregated by config	Section IV, Fig. 6 (existing)
New Fig: PPO vs A* per-map comparison	results.csv pivoted by method	Section IV-G (new)
New Fig: reward sensitivity heatmap	Sensitivity sweep aggregated	Section IV-F (new)
Table: configuration summary	config.json files	Section III-C (new)
Table: compute budget	wall_clock_sec fields	Section IV-H (new)
Table: paired statistical tests	results.csv + scipy	Section IV-G (new)

10. Troubleshooting

10.1 Symptom: "no kernel image is available for execution on the device"

Cause: PyTorch wheel does not include sm_120 kernels (you have cu118 / cu121 / cu124 instead of cu128). Fix: reinstall torch from the cu128 index URL (Section 1.2). Verify with the stack check.

10.2 Symptom: training runs but GPU-Util is 0-5%

Cause 1: device defaulted to CPU. Print `model.device` after constructing PPO; it must be `cuda:0`. Cause 2: only one environment is being stepped (no SubprocVecEnv); the env-stepping bottleneck masks GPU activity. Use 16-env vectorisation as in Section 4.1.

10.3 Symptom: "CUDA out of memory" with one run

Unlikely on a 32 GB 5090 with this architecture. If it happens, you probably left stale tensor handles -- run only one PPO process at a time and confirm `nvidia-smi` shows zero VRAM use before launching.

10.4 Symptom: episode reward starts high then collapses

Indicates entropy collapse or value-function explosion. Confirm `ent_coef = 0.05` (not zero). Confirm reward magnitudes match Table III exactly; a stray +1000 from legacy code will dominate the gradient.

10.5 Symptom: episode reward never rises

Confirm the bounding-area reward is being assigned (set a breakpoint in step() and print). Confirm at least 50% of episodes finish in goal -- if every episode ends in timeout, the threshold may be unreachable for the current map size; lower it to 16 for 20×20 and 25 for 25×25.

10.6 Symptom: A baseline beats PPO on every map*

Not a bug -- it can happen if the meeting-point heuristic is well-chosen for the specific maps. Report it honestly. The contribution framing ("no meeting-point heuristic required") still holds.

10.7 Symptom: 4 parallel processes are slower than sequential

CPU contention from too many environment workers. Reduce per-run `n_envs` from 16 to 8, or drop concurrency from 4 to 2.

11. End-to-end runbook

Execute these steps in order. Each step assumes the previous succeeded.

#	Step	Estimated ti
1	Pre-flight: run python scripts/check_stack.py. Confirm STACK OK, sm_120 detected.	~10 min
2	Write env_revision.py and map_revision.py. Unit-test with a 100-	~3 h
3	Write train_revision.py. Smoke-test with --n-steps 5000 --n-robots 2 --n-en	~2 h
4	Write astar_baseline.py and eval_revision.py; smoke-test by rolli	~3 h
5	Touch up reward_sensitivity.py to accept --n-envs.	~30 min
6	Launch full training: powershell -File run_all_revision.ps1. Trains 4 PPO models (separate	~12 h (4 parallel processes, 5 h each)
7	Launch python eval_revision.py --tag primary --episodes 20 --baseline astar. Emits results.cs	~1 h
8	Launch python plots_revision.py --results-csv results.csv --out-dir results_revision/figures/. E	~5 min
9	Drop the new figures into Overleaf; replace the placeholder text in root_revised.tex sections I	~2 h
10	Re-compile root_revised.tex; visual diff vs. the original to confirm the changes look right.	~10 min
11	Draft the rebuttal letter from Section 1 of Revision_Plan.pdf in third-person form.	~3 h
12	Submit.	

Critical-path summary: the entire revision (code, training, evaluation, figures, paper update, rebuttal) fits in **~3 working days** on this 5090 workstation, with most of the wall-clock spent on writing rather than computing.

End of runbook.